

Environment config

COMPLETE LEARNING GUIDE

What You'll Learn

The same application code typically needs to run against different configurations in different places — a local database on a developer's laptop, a staging database, a production database with real credentials — and hardcoding any of those values directly into source code is one of the fastest ways to leak a secret or accidentally point a local test run at a production database. Environment configuration is the practice of keeping every value that differs between environments (connection strings, API keys, feature flags, ports) outside the codebase entirely, injected at runtime through environment variables. This guide covers the `.env` file pattern, validating required configuration at startup so misconfiguration fails loudly instead of silently, and the secrets-management discipline that keeps real credentials out of version control and out of the hands of anyone who shouldn't have them.

1. The Twelve-Factor Principle: Config Lives in the Environment

The widely referenced 'twelve-factor app' methodology states plainly that configuration — anything likely to vary between deployments — should be stored in environment variables, strictly separate from the code. The test it proposes is simple: could this codebase be made open source right now without exposing any secret? If a database password or API key is sitting in a config file checked into the repository, the answer is no. Environment variables satisfy this cleanly because they live outside the code entirely, are read at process startup, and can differ freely between a developer's laptop, CI, staging, and production without a single line of code changing.

2. `.env` Files and `dotenv`

In local development, environment variables are conventionally kept in a `.env` file at the project root — a simple `KEY=value` list — which a library like `dotenv` loads into `process.env` when the application starts. This file must never be committed to version control (it should be listed in `.gitignore` from day one), since it typically holds real local credentials. In staging and production, the actual values are instead injected by the hosting platform itself (a cloud provider's environment variable settings, a Kubernetes secret, a CI/CD pipeline's secret store) rather than from a `.env` file, which usually doesn't even exist in those environments.

BASH

```
# .env (local development only – never commit this file)
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://user:password@localhost:5432/myapp_dev
JWT_SECRET=replace_with_a_long_random_value
SESSION_SECRET=replace_with_another_long_random_value
STRIPE_API_KEY=sk_test_XXXXXXXXXXXX
```

3. Loading and Validating Configuration at Startup

Reading `process.env.DATABASE_URL` scattered throughout the codebase, wherever it happens to be needed, makes it hard to know what configuration the app actually depends on and easy to typo a variable name with no error at all — a mistyped key simply reads as undefined silently. The better pattern is a single config module that reads every environment variable once, validates that every required one is actually present, and exports a clean, typed configuration object that the rest of the app imports — failing fast and loudly at startup if anything required is missing, rather than failing confusingly later when that missing value is first used.

JAVASCRIPT

```
require("dotenv").config();

const requiredEnvVars = ["DATABASE_URL", "JWT_SECRET", "SESSION_SECRET"];
for (const key of requiredEnvVars) {
  if (!process.env[key]) {
    throw new Error(`Missing required environment variable: ${key}`);
  }
}

const config = {
  env: process.env.NODE_ENV || "development",
  port: parseInt(process.env.PORT, 10) || 3000,
  databaseUrl: process.env.DATABASE_URL,
  jwtSecret: process.env.JWT_SECRET,
};

module.exports = config;
```

4. Different Configuration Per Environment

Beyond secrets, plenty of non-sensitive configuration also legitimately differs by environment: log verbosity (verbose in development, warnings-and-errors-only in production), which external services to call (a sandbox payment API in staging, the real one in production), and feature flags controlling in-progress features. Reading `process.env.NODE_ENV` (or an equivalent) to branch this kind of behavior is standard practice, but it should be used narrowly for genuinely environment-specific behavior — not as a general-purpose if-statement scattered through business logic, which makes the code's actual behavior harder to reason about in any single environment.

5. Secrets Management Beyond .env Files

A .env file is adequate for local development but is not a real secrets-management solution for production — it has no access control, no rotation mechanism, and no audit trail of who read which secret when. Production systems typically use a dedicated secrets manager (AWS Secrets Manager, HashiCorp Vault, or a cloud platform's built-in encrypted environment variable store) that injects secrets into the running process at deploy time, supports rotating a compromised credential without a code change, and logs access. Even without a dedicated secrets manager, the baseline discipline — never commit a real secret, never log a secret value, never put a secret in a URL query string — applies everywhere.

- Never commit .env files (or any file with real credentials) to version control — enforce this with .gitignore from the project's first commit.
- Rotate a secret immediately if it's ever accidentally exposed (committed, logged, pasted somewhere public) — don't just remove it and assume it's safe.
- Use different secrets per environment — a compromised staging credential should never also grant access to production.
- Avoid logging full configuration objects that might include secret values, even for debugging.

6. The .env.example Pattern

New developers joining a project need to know which environment variables exist and roughly what they should look like, without ever seeing real secret values. The standard solution is a committed .env.example file mirroring the real .env's keys with placeholder or dummy values (JWT_SECRET=replace_me instead of a real secret), giving a new contributor a copy-pasteable starting point and giving the codebase a single, version-controlled source of truth for 'what configuration does this app need.'

Common Pitfalls

- Committing a .env file containing real secrets to version control, often permanently exposing them in the repository's history even after later deletion.
- Hardcoding an API key or database password directly in source code instead of reading it from configuration.
- Not maintaining a .env.example file, leaving new developers to guess which environment variables the application actually requires.
- Using the same secrets (database password, API keys) across development, staging, and production instead of distinct ones per environment.
- Letting a missing required environment variable default silently to an empty string or undefined instead of crashing loudly at startup.
- Logging full configuration or request objects that happen to include secret values, leaking them into log aggregation systems.

Key Takeaways

- Configuration that varies between environments belongs in environment variables, never hardcoded in source code.
- .env files are for local development only — production configuration should come from the hosting platform's own secret/config store.
- Validate that all required environment variables are present at application startup, so misconfiguration fails immediately and loudly, not confusingly later.
- A committed .env.example file (with placeholder values) documents required configuration without exposing any real secret.
- Use distinct secrets per environment, and rotate any secret immediately if it's ever accidentally exposed.

Learn More

The Twelve-Factor App — Config

<https://12factor.net/config>

dotenv — npm package documentation

<https://www.npmjs.com/package/dotenv>

OWASP — Secrets Management Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html